XML, XPath, JSON y MongoDB

Datos estructurados, consultas y fundamentos de API

CódigoClaro · Material didáctico de apoyo · Edición octubre de 2026

Cómo estudiar este material

Lee el concepto, escribe los ejemplos sin copiar y modifica los casos de prueba. Los fragmentos son ejemplos didácticos: adapta nombres, tipos y validaciones al enunciado real.

1. XML bien formado

XML representa datos mediante etiquetas anidadas. Debe existir un único elemento raíz, etiquetas correctamente cerradas y atributos entre comillas. XML bien formado no significa necesariamente válido respecto a un esquema.

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
  <libro id="b1">
    <titulo>Java</titulo>
    <precio moneda="EUR">24.95</precio>
  </libro>
</catalogo>
```

2. XPath esencial

```
/catalogo/libro                seleccionar libros directos
//libro                         seleccionar libros en cualquier nivel
/catalogo/libro[@id="b1"]      filtrar por atributo
//libro[titulo="Java"]         filtrar por hijo
//libro/precio/text()           obtener el texto del precio
count(//libro)                  contar libros
```

/ selecciona desde la raíz; // busca descendientes; @ selecciona atributos; [ ] aplica predicados. La posición en XPath suele empezar en 1, a diferencia de muchos lenguajes de programación.

3. Ejes y funciones

```
//libro/parent::*
//libro/ancestor::catalogo
//libro[price > 20]
//libro[contains(titulo, "Java")]
//libro[position() = 1]
```

Practica parent, child, ancestor, descendant, following-sibling y preceding-sibling. Recuerda que namespaces pueden exigir prefijos al consultar XML con espacios de nombres.

4. XSD y validación

DTD y XML Schema permiten definir qué estructura y tipos se consideran válidos. XSD soporta tipos más ricos, restricciones y espacios de nombres. La validación detecta datos que no cumplen el contrato esperado.

5. JSON y API REST

```
{
  "id": 1,
  "nombre": "Curso Java",
  "activo": true
}
```

```
GET /api/cursos       consultar
POST /api/cursos      crear
PUT /api/cursos/1     reemplazar/actualizar
DELETE /api/cursos/1  eliminar
```

JSON utiliza objetos, arrays, cadenas, números, booleanos y null. Una API REST intercambia recursos mediante HTTP. Comprueba códigos de estado, valida entradas y no expongas secretos en el cliente.

6. MongoDB: introducción

```
db.cursos.insertOne({ nombre: "Java", nivel: "inicial" })
db.cursos.find({ nivel: "inicial" })
db.cursos.updateOne(
  { nombre: "Java" },
  { $set: { nivel: "intermedio" } }
)
```

MongoDB almacena documentos BSON en colecciones. Diseña documentos según los patrones de consulta; decide entre anidar o referenciar según tamaño, consistencia y acceso. Usa índices cuando las consultas los justifiquen.

Ejercicios

- Escribe una expresión XPath para obtener todos los libros con precio superior a 20.
- Distingue XML bien formado de XML válido.
- Diseña una API CRUD para cursos y define sus códigos de respuesta.
- Escribe una consulta MongoDB que encuentre cursos activos de nivel inicial.